Implementacja sieci neuronowej MLP i ewaluacja w zadaniach klasyfikacji i regresji

Hubert Błonowski Jan Zaborski

Streszczenie

Zaimplementowaliśmy od podstaw perceptron wielowarstwowy (MLP) uczony metodą propagacji wstecznej i stochastycznego spadku gradientu. Na dwóch zbiorach do klasyfikacji punktów na płaszczyźnie i dwóch zbiorach do regresji funkcji jednej zmiennej zbadaliśmy wpływ funkcji aktywacji, liczby i szerokości warstw ukrytych, funkcji kosztu oraz biasów na jakość uczenia. Każdą konfigurację uczono z pięcioma ziarnami losowymi. Tę samą implementację zastosowaliśmy do rozpoznawania cyfr ze zbioru MNIST, osiągając 98,3% trafności na zbiorze testowym siecią z dwiema warstwami ukrytymi po 512 neuronów.

Spis treści

1 Wstęp **2**

2 Podstawy teoretyczne **2**

3 Implementacja sieci **3**

4 Procedura eksperymentalna **4**

5 Wyniki: klasyfikacja w $\mathbb{R}^2$ i regresja $\mathbb{R} \to \mathbb{R}$ **5**
5.1 Wizualizacja uczenia 5
5.2 Architektura sieci 5
5.3 Funkcja aktywacji 11
5.4 Funkcja kosztu 12
5.5 Biasy 12

6 Rozpoznawanie cyfr MNIST **16**
6.1 Dane i procedura 16
6.2 Architektura 16
6.3 Funkcja aktywacji i funkcja kosztu 16
6.4 Biasy 18
6.5 Analiza błędów i wag 18

7 Podsumowanie i wnioski **20**

1 Wstęp

Rozważamy dwa podstawowe zadania uczenia nadzorowanego. W *klasyfikacji* na podstawie wektora cech należy wskazać jedną z kilku kategorii, np. przypisać punkt płaszczyzny do jednej z klas lub obraz cyfry do cyfry, którą przedstawia. W *regresji* należy przewidzieć wartość liczbową, np. wartość nieznanej funkcji w danym punkcie. W obu przypadkach model uczy się na zbiorze przykładów z znanymi odpowiedziami (zbiór uczący), a jego jakość ocenia się na osobnym zbiorze testowym, którego nie widział podczas uczenia.

Celem projektu jest implementacja sieci MLP do obu zadań, wizualizacja jej działania i uczenia oraz zbadanie, jak na wyniki wpływają funkcja aktywacji, architektura sieci i funkcja kosztu. W części pierwszej korzystamy z małych zbiorów danych w $\mathbb{R}^2$ (klasyfikacja) i $\mathbb{R}$ (regresja), które pozwalają wprost zobaczyć, czego nauczyła się sieć. W części drugiej tę samą implementację stosujemy do zbioru odręcznie pisanych cyfr MNIST.

Rozdział 2 krótko omawia ideę sieci MLP i ich uczenia, rozdział 3 naszą implementację, a rozdział 4 procedurę eksperymentalną. Wyniki dla małych zbiorów przedstawia rozdział 5, dla MNIST rozdział 6. Szczegółowe wzory pomijamy, odsyłając do literatury.

2 Podstawy teoretyczne

Podstawowym elementem sieci MLP (*Multi-Layer Perceptron*) jest neuron o wejściach $x_1, \ldots, x_n$, wagach $w_1, \ldots, w_n$, biasie b i funkcji aktywacji f, obliczający $y = f(\sum_{i=1}^{n} w_i x_i + b)$. Neurony przetwarzające to samo wejście tworzą warstwę (zakładamy, że mają wspólną funkcję aktywacji). Sieć składa się z warstw ukrytych i warstwy wyjściowej, przy czym wejściem każdej warstwy jest wyjście poprzedniej, a obliczenie predykcji przez kolejne warstwy nazywamy przejściem w przód (*forward pass*).

Funkcje aktywacji warstw ukrytych (np. sigmoida, tanh, ReLU) muszą być nieliniowe, bo złożenie przekształceń liniowych jest liniowe i głębsza sieć nie dawałaby nic więcej niż jedna warstwa. W warstwie wyjściowej stosuje się identyczność w regresji oraz softmax w klasyfikacji, zwracający prawdopodobieństwa przynależności do każdej z klas.

Jakość predykcji $\hat{y}$ względem wartości prawdziwej y mierzy funkcja kosztu $\mathcal{L}(y, \hat{y})$, np. błąd średniokwadratowy (MSE) lub średni błąd bezwzględny (MAE) w regresji oraz entropia krzyżowa w klasyfikacji. Uczenie polega na minimalizacji średniego kosztu na zbiorze uczącym metodą spadku gradientu: wagi i biasy są wielokrotnie przesuwane o mały krok (współczynnik uczenia) w kierunku przeciwnym do gradientu kosztu. W wersji stochastycznej (SGD) gradient liczy się na małych, losowych porcjach danych (*mini-batch*), co pozwala wykonać znacznie więcej kroków w tym samym czasie. Wagi inicjalizuje się losowo, aby

neurony jednej warstwy nie uczyły się identycznie.

Gradient względem wszystkich parametrów wyznacza algorytm propagacji wstecznej (*backpropagation*), oparty na regule łańcuchowej: błąd $\delta = \partial\mathcal{L}/\partial z$ liczony względem wejść neuronów wyjściowych (przed aktywacją) jest przekazywany warstwa po warstwie w stronę wejścia, a w każdej warstwie wyznacza się z niego gradienty jej wag. Koszt obliczeń jest tego samego rzędu co jedno przejście w przód.

Wszystkie te operacje można zapisać w języku algebry liniowej. Wagi warstwy tworzą macierz, a porcja danych macierz, której wiersze są przykładami; przejście w przód przez warstwę jest wtedy jednym mnożeniem macierzy, a propagacja wsteczna sprowadza się do mnożeń przez macierze transponowane. Mnożenie macierzy wykonuje dużo obliczeń na każdej wczytanej liczbie, dzięki czemu dobrze wykorzystuje pamięć podręczną, instrukcje wektorowe procesora (SIMD) i karty graficzne.

3 Implementacja sieci

Sieć oraz wszystkie operacje na macierzach zaimplementowano od podstaw w języku Rust, bez bibliotek uczenia maszynowego i algebry liniowej. Jedyną zewnętrzną zależnością biblioteki jest generator liczb losowych. Rust wybrano ze względu na niskopoziomową kontrolę nad układem danych i obliczeniami, wygodną składnię oraz automatyczną wektoryzację kodu przez kompilator. W kodzie nie używano jawnych instrukcji SIMD ani wielowątkowości, mimo to wydajność jest zadowalająca: epoka uczenia na MNIST trwa od ok. 1 s (sieć 784–128–10) do ok. 10 s (784–512–512–10), wliczając ewaluację na zbiorach uczącym i testowym.

Macierze przechowywane są wierszowo (row-major), a porcja danych jest macierzą, której wiersze są przykładami. Warstwa liczy więc $Z = XW + b$, gdzie W ma wymiary *wejścia* $\times$ *neurony*, czyli jest transpozycją macierzy wag z typowego zapisu w literaturze. Układ ten zapewnia ciągły dostęp do pamięci zarówno przy przetwarzaniu przykładów, jak i wierszy wag.

Sieć to lista warstw (macierz wag i opcjonalny wektor biasów) wraz ze wspólną funkcją aktywacji warstw ukrytych oraz funkcją aktywacji wyjścia połączoną z funkcją kosztu. Przejście w przód składa się z mnożenia macierzy (GEMM), dodania biasu do każdego wiersza i zastosowania funkcji aktywacji element po elemencie. Wyniki pośrednie zapisywane są w przydzielanej raz pamięci roboczej i wykorzystywane podczas propagacji wstecznej. Gradienty wag i błąd przekazywany do poprzedniej warstwy liczone są wariantami GEMM, które traktują jeden z argumentów jako transponowany, więc transpozycji nie trzeba tworzyć w pamięci. Wagi inicjalizowane są losowo metodą Xaviera [1] (dla ReLU metodą He [2]), biasy zerami, a wejścia są normalizowane statystykami zbioru uczącego.

Każdy eksperyment opisuje plik JSON z danymi, architekturą (lista szerokości warstw ukrytych, obecność biasów, funkcja aktywacji, funkcja kosztu), parametrami uczenia i listą ziaren. Ziarno inicjalizuje generator, z którego loso-

wane są wagi początkowe i kolejność przykładów w każdej epoce, więc uczenie z tym samym plikiem i ziarnem jest w pełni powtarzalne. Program zapisuje konfigurację razem z wynikami, a dla każdego ziarna: wartości kosztu i metryk w każdej epoce, predykcje na zbiorze testowym (dla danych jedno- i dwuwymiarowych) oraz opcjonalnie wagi wszystkich warstw i sygnały δ propagowane wstecz dla ustalonego przykładu uczącego. Na tej podstawie skrypty w Pythonie rysują wykresy z rozdziałów 5–6 oraz animację ewolucji wag w postaci grafu sieci.

4 Procedura eksperymentalna

Zbiory danych. Zbiory *simple* i *three_gauss* zawierają punkty płaszczyzny podzielone odpowiednio na 2 i 3 klasy; w *simple* klasy są rozdzielone prawie liniowo, a w *three_gauss* pochodzą z trzech częściowo nakładających się rozkładów normalnych. Zbiory *cube* i *activation* zawierają próbki funkcji $\mathbb{R} \to \mathbb{R}$: wielomianu stopnia trzeciego i funkcji o kształcie esowatym. W obu zbiorach regresyjnych punkty testowe pokrywają szerszy przedział x niż uczące (tab. 1), dlatego błąd testowy liczymy osobno dla punktów z zakresu danych uczących (interpolacja) i spoza niego (ekstrapolacja). Zbiór MNIST opisuje rozdział 6.

	simple	*three_gauss*	*cube*	*activation*
zadanie	klasyfikacja w $\mathbb{R}^2$		regresja $\mathbb{R} \to \mathbb{R}$	
liczba klas / wyjść	2	3	1	1
przykłady: uczące / testowe	1000 / 1000	1000 / 1000	1000 / 1000	1000 / 1000
zakres x: uczący / testowy	—	—	$[-2, 4]$ / $[-5, 5]$	$[-4, 4]$ / $[-5, 5]$
test w zakresie / poza zakresem	—	—	599 / 401	796 / 204

Tabela 1: Zbiory danych części pierwszej.

Plan badań. Każde badanie zmienia jeden czynnik względem konfiguracji bazowej (tab. 2):

- architekturę: liczbę warstw ukrytych $L \in \{0, \ldots, 4\}$ i liczbę neuronów w warstwie $B \in \{4, 8, 16, 32, 64\}$ (wszystkie kombinacje),
- funkcję aktywacji warstw ukrytych: sigmoida, tanh, ReLU,
- funkcję kosztu: entropia krzyżowa i MSE (na wyjściu softmax) dla klasyfikacji, MSE i MAE dla regresji,
- obecność biasów, dla sieci 1×8 i 2×32.

Ponieważ najlepszy współczynnik uczenia zależy od funkcji aktywacji i kosztu, oba te badania powtórzono dla współczynników {0,003; 0,01; 0,03; 0,1; 0,3; 1}.

Ocena. Hiperparametrów nie dobieraliśmy na podstawie zbioru testowego: sieć uczy się na całym zbiorze uczącym przez stałą liczbę epok, a zbiór testowy jest oceniany po każdej epoce wyłącznie w celu zapisu wyników. Mierzymy trafność (accuracy) i macro-F1 w klasyfikacji oraz MSE i MAE (w oryginalnych jednostkach y) w regresji. Ponieważ przy stałym współczynniku uczenia wynik waha się między epokami, jako wynik końcowy przyjmujemy średnią z ostatnich 50 epok. Każdą konfigurację uczymy z 5 ziarnami (0–4) i podajemy medianę oraz rozstęp międzykwartylowy (IQR). Przebiegi, w których uczenie się rozbiegło (wartości nieskończone), pomijamy w statystykach i zaznaczamy na wykresach symbolem $\times n$, gdzie n to liczba takich ziaren.

	klasyfikacja	regresja
warstwy ukryte $L \times B$	2×32	
aktywacja warstw ukrytych	tanh	
wyjście i funkcja kosztu	softmax, entropia krzyżowa	liniowe, MSE
biasy / inicjalizacja	tak / Xavier, biasy zerowe	
optymalizator	mini-batch SGD, stały współczynnik 0,1, batch 32	
liczba epok	260 (*simple*), 340 (*three_gauss*)	1000
normalizacja	wejścia: standaryzacja	wejścia i wyjście: standaryzacja
precyzja obliczeń	64-bitowa	

Tabela 2: Konfiguracja bazowa części pierwszej. Każde badanie zmienia jeden z czynników, pozostałe pozostają bez zmian.

5 Wyniki: klasyfikacja w $\mathbb{R}^2$ i regresja $\mathbb{R} \to \mathbb{R}$

5.1 Wizualizacja uczenia

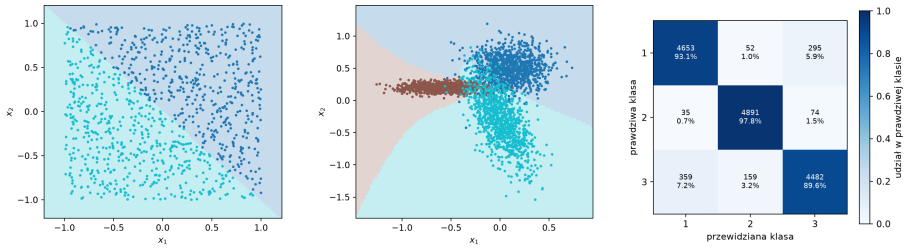
Rysunek 1 przedstawia obszary decyzyjne sieci bazowej. Na *simple* sieć myli się tylko przy granicy klas, a na *three_gauss* w obszarach, gdzie klasy się nakładają. Rysunek 2 pokazuje funkcje nauczone w regresji: w zakresie danych uczących sieć pokrywa się z funkcją docelową, a poza nim jej przebieg zależy od funkcji aktywacji.

Przebieg kosztu w kolejnych epokach przedstawia rys. 3. Koszt na zbiorze testowym przez cały czas uczenia pozostaje blisko kosztu na zbiorze uczącym. Na *simple* i *cube* koszt wyraźnie oscyluje między epokami.

Ewolucję wag i propagowanego błędu pokazujemy dla mniejszej sieci 2×8 na zbiorze *cube* (rys. 4–6). Wagi zmieniają się najszybciej na początku uczenia, kiedy najszybciej maleje też koszt. Sygnały δ we wszystkich warstwach są największe (do ok. 0,6 co do wartości bezwzględnej) w pierwszych kilkudziesięciu epokach, a po ok. 400 epokach są bliskie zeru.

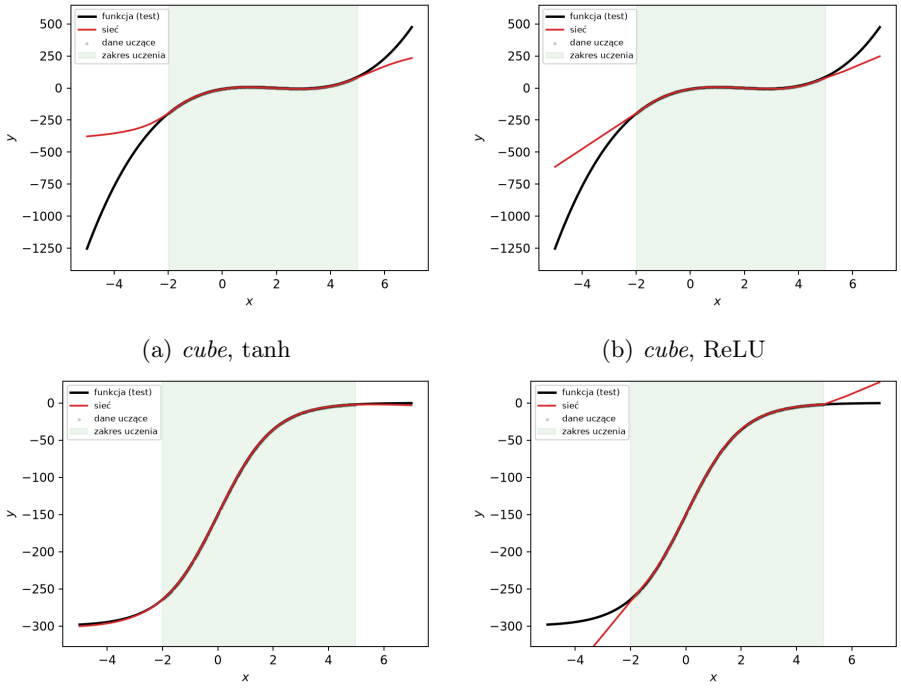
5.2 Architektura sieci

Wyniki dla wszystkich architektur przedstawia rys. 7. W klasyfikacji architektura ma niewielki wpływ. Na *simple* najlepsza jest sieć bez warstw ukrytych



(a) *simple* (b) *three_gauss* (c) *three_gauss*

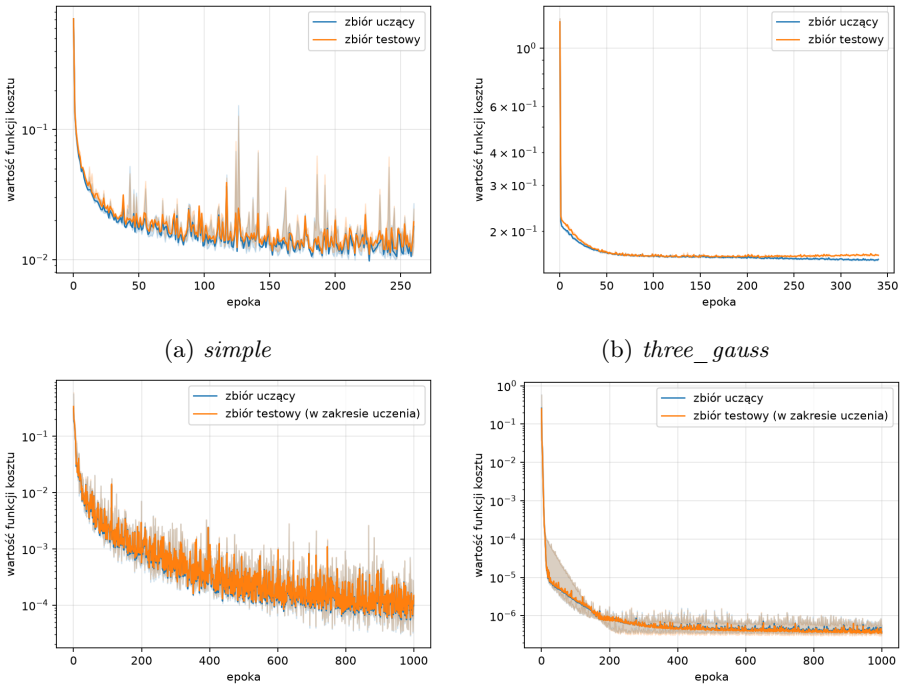
Rysunek 1: Klasyfikacja siecią bazową. (a), (b): tło – klasa przewidziana przez sieć, punkty – zbiór testowy w kolorze prawdziwej klasy, × – pomyłka (ziarno 0). (c): macierz pomyłek na zbiorze testowym zsumowana po ziarnach.



(a) *cube*, tanh (b) *cube*, ReLU

(c) *activation*, tanh (d) *activation*, ReLU

Rysunek 2: Regresja siecią 2×32 (ziarno 0) na całym zakresie zbioru testowego. Zielone tło oznacza zakres x danych uczących.



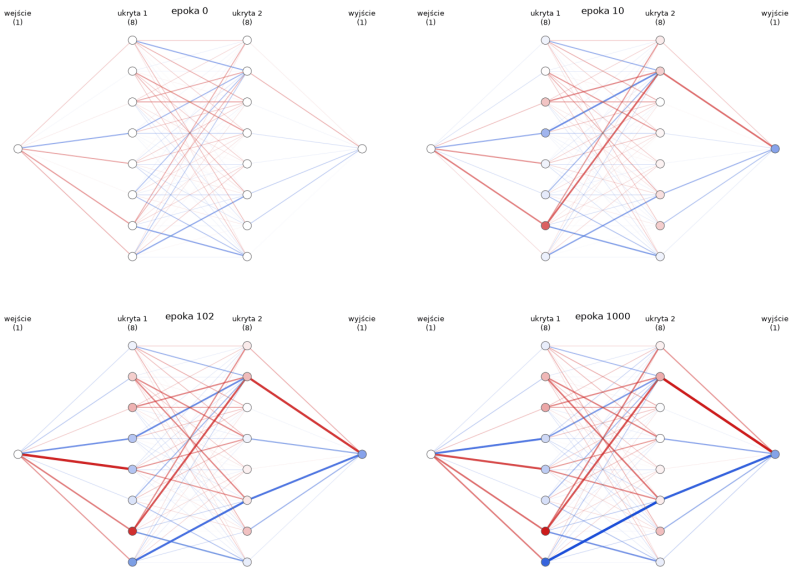
(a) *simple*

(b) *three_gauss*

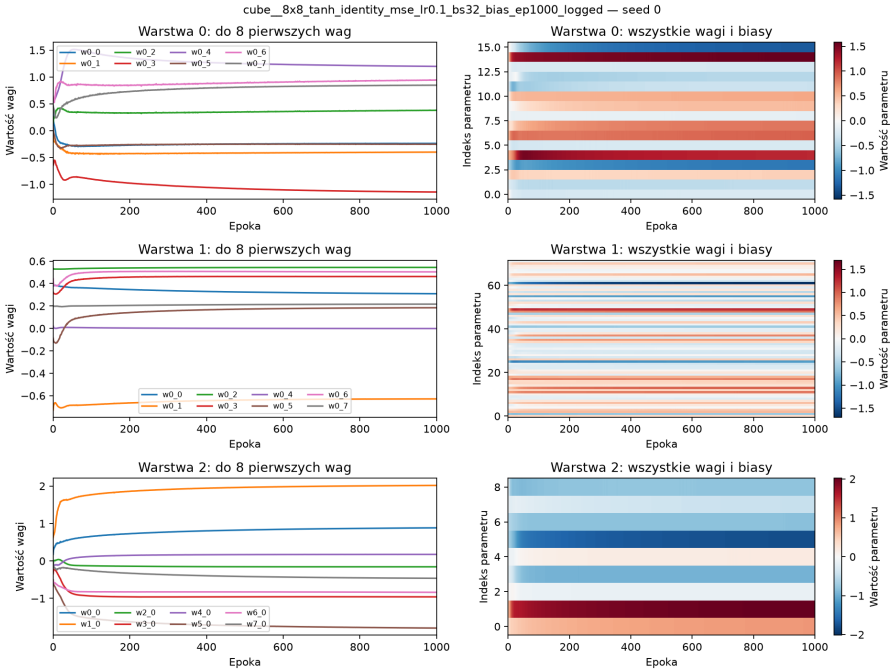
(c) *cube*

(d) *activation*

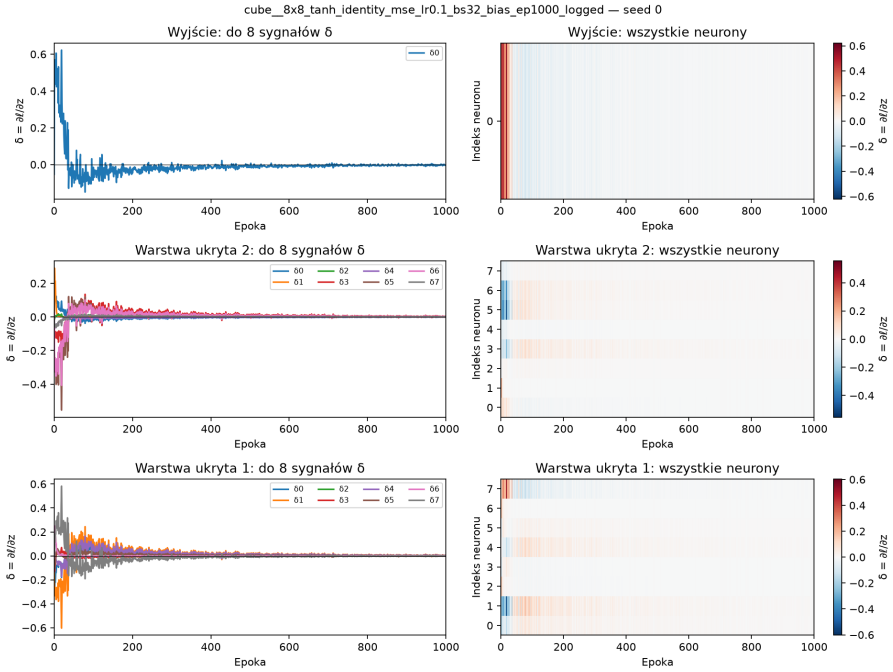
Rysunek 3: Wartość funkcji kosztu sieci bazowej w kolejnych epokach (mediana i IQR z 5 ziaren). W regresji koszt liczony jest dla znormalizowanych wartości y.



Rysunek 4: Wagi sieci 2×8 (*cube*, ziarno 0) w epokach 0, 10, 100 i 1000. Kolor krawędzi to znak wagi (czerwony – dodatnia, niebieski – ujemna), grubość i nasycenie – jej wartość bezwzględna; wypełnienie neuronu oznacza bias. Każda warstwa ma własną skalę. Pełna animacja: `weights_graph.gif` w katalogu wyników.



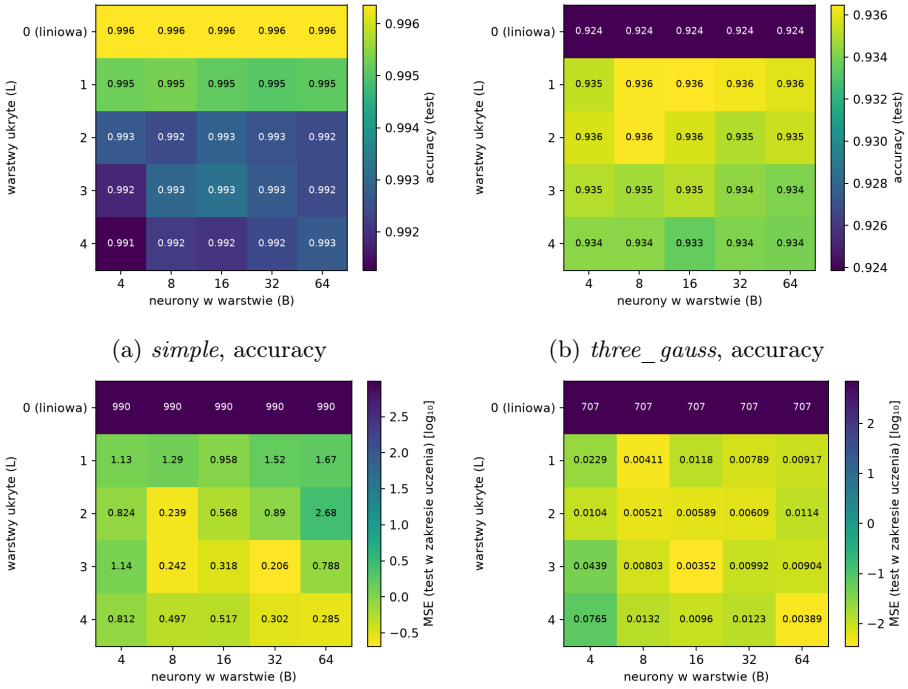
Rysunek 5: Wartości wag sieci 2×8 (*cube*, ziarno 0) po kolejnych epokach. Z lewej przebieg 8 wybranych wag każdej warstwy, z prawej wszystkie wagi i biasy warstwy.



Rysunek 6: Błąd $\delta = \partial \mathcal{L}/\partial z$ propagowany wstecz od wyjścia (u góry) do pierwszej warstwy ukrytej (u dołu), dla ustalonego przykładu uczącego po każdej epoce (*cube*, sieć 2×8, ziarno 0).

(99,6%), a sieci głębsze osiągają 99,1–99,5%. Na *three_gauss* sieć liniowa osiąga 92,4%, a każda sieć z warstwami ukrytymi 93,4–93,7%; sieci z 3–4 warstwami mają wyższą trafność na zbiorze uczącym niż testowym (np. 4×16: 93,8% wobec 93,4%).

W regresji sieć bez warstw ukrytych nie jest w stanie dopasować funkcji (MSE 990 na *cube* i 707 na *activation*), natomiast już jedna warstwa ukryta zmniejsza błąd o 3–5 rzędów wielkości. Na *cube* sieci z 2–4 warstwami są zwykle lepsze niż z jedną (MSE 0,2–0,9 wobec 1,0–1,7). Poza zakresem uczenia (rys. 8) błąd na *cube* rośnie z liczbą warstw, a dla sieci z jedną warstwą maleje z jej szerokością.



(a) *simple*, accuracy

(b) *three_gauss*, accuracy

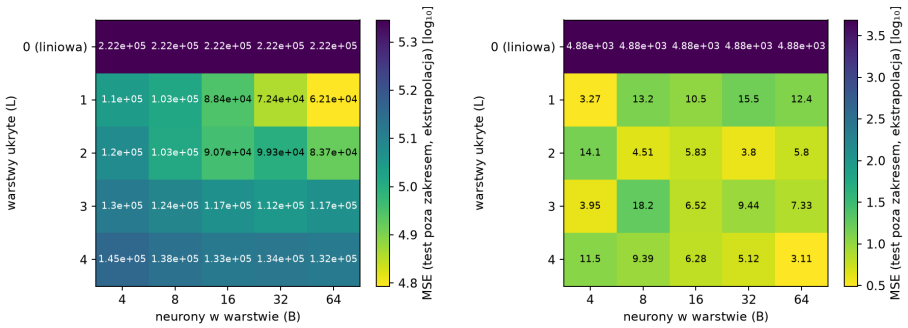
(c) *cube*, MSE w zakresie uczenia

(d) *activation*, MSE w zakresie uczenia

Rysunek 7: Wynik na zbiorze testowym w zależności od liczby warstw ukrytych L i liczby neuronów w warstwie B (mediana z 5 ziaren). W regresji kolor odpowiada $\log_{10}$ MSE.

5.3 Funkcja aktywacji

Wpływ funkcji aktywacji silnie zależy od współczynnika uczenia (rys. 9). Sigmoida wymaga największego kroku: na *cube* osiąga MSE 0,61 przy współczynniku 0,3, a 241 przy 0,01, a na *three_gauss* jest najlepsza przy 0,3–1. tanh i ReLU osiągają najlepsze wyniki przy mniejszych krokach, ale w regresji przy dużym



(a) *cube* (b) *activation*

Rysunek 8: MSE poza zakresem danych uczących w zależności od architektury.

kroku uczenie się rozbiega: przy współczynniku 1 dla tanh we wszystkich ziarnach, dla ReLU w 3–5 ziarnach. Sigmoida nie rozbiegła się w żadnym przypadku. Przy najlepszym kroku różnice między aktywacjami w klasyfikacji nie przekraczają 0,3 p.p., natomiast w regresji są duże: na *activation* tanh osiąga MSE 0,006, a sigmoida i ReLU ok. 0,25.

Funkcja aktywacji decyduje też o ekstrapolacji (rys. 2, 10). Na *cube* najmniejszy błąd poza zakresem uczenia daje ReLU (MSE ok. $5 \cdot 10^4$ wobec ok. 10^5 dla tanh i sigmoidy), a na *activation* tanh (ok. 4 wobec 160–230 dla sigmoidy i 1400–1800 dla ReLU).

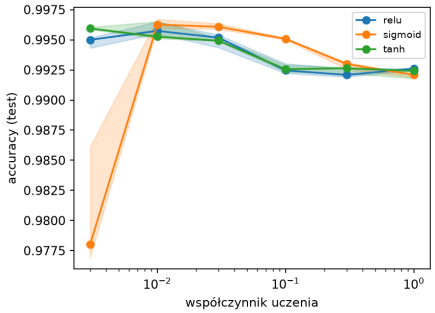
5.4 Funkcja kosztu

W klasyfikacji (rys. 11a,b) MSE na wyjściu softmax wymaga większego kroku niż entropia krzyżowa: na *three_gauss* najlepsze są odpowiednio współczynniki 0,1–0,3 i 0,03, przy podobnej najlepszej trafności (93,6–93,7%). Na *simple* najlepsze wyniki obu funkcji to 99,6–99,7%.

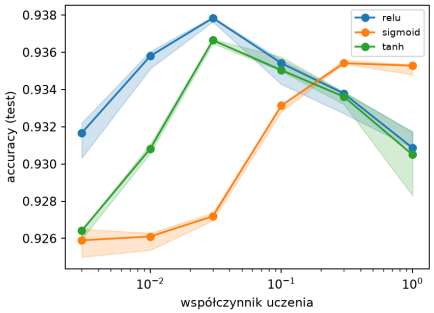
W regresji (rys. 11c,d) koszt MAE daje gorsze wyniki niż MSE nawet w mierze MAE: na *cube* najlepsze MAE wynosi 2,1 przy uczeniu z kosztem MAE i 0,68 z kosztem MSE, na *activation* odpowiednio 0,68 i 0,07. Koszt MAE daje najlepsze wyniki przy najmniejszych badanych krokach (0,003–0,01).

5.5 Biasy

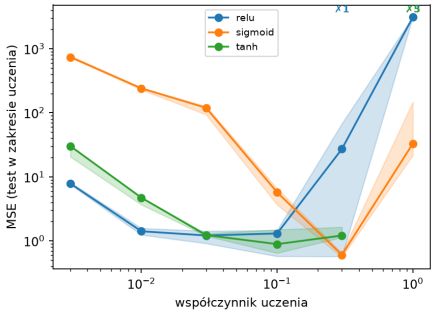
Usunięcie biasów (rys. 12) obniża trafność o 0,15–0,3 p.p. na *simple* i o 1,1–1,4 p.p. na *three_gauss*. W regresji sieć bez biasów praktycznie nie uczy się funkcji: MSE w zakresie uczenia wynosi 420–520 na *cube* i ok. 660 na *activation*, wobec 0,004–1,3 z biasami.



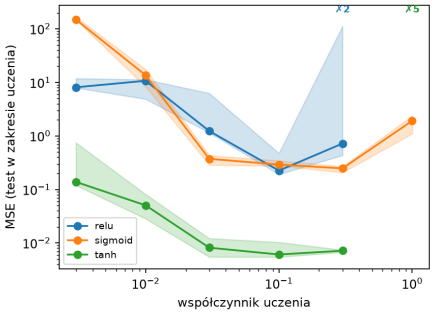
(a) *simple*, accuracy



(b) *three_gauss*, accuracy

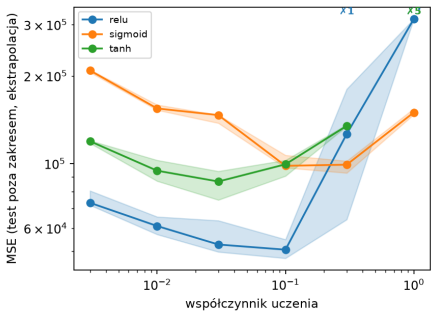


(c) *cube*, MSE w zakresie uczenia

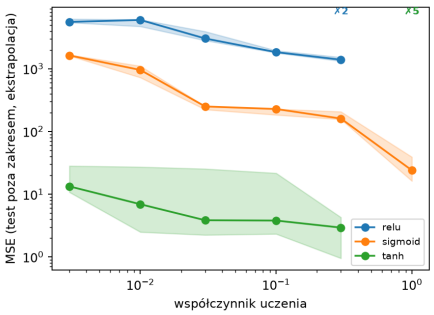


(d) *activation*, MSE w zakresie uczenia

Rysunek 9: Funkcja aktywacji warstw ukrytych w zależności od współczynnika uczenia (sieć 2×32).

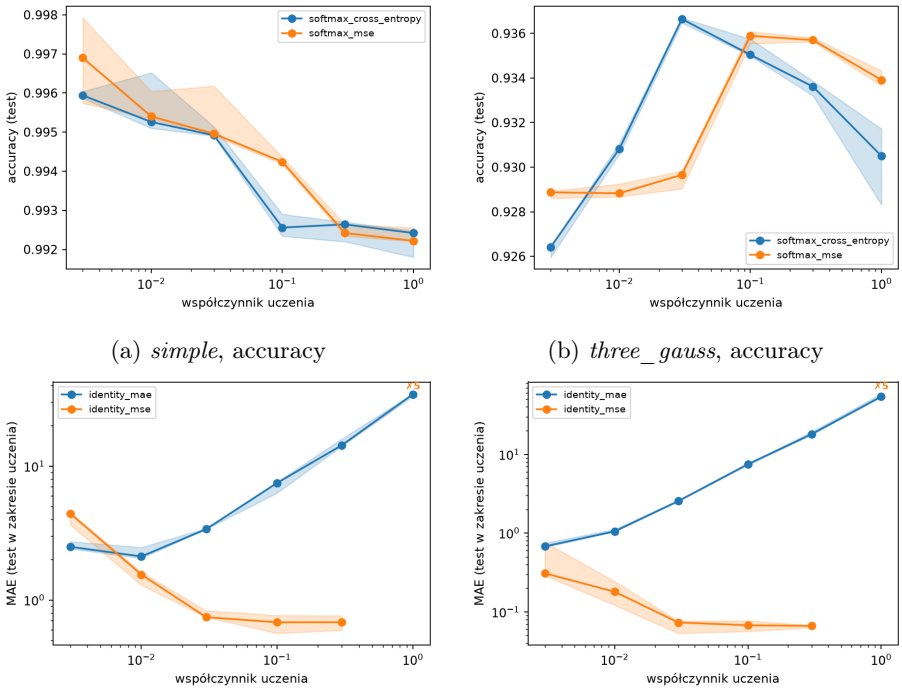


(a) *cube*



(b) *activation*

Rysunek 10: MSE poza zakresem danych uczących dla różnych funkcji aktywacji.



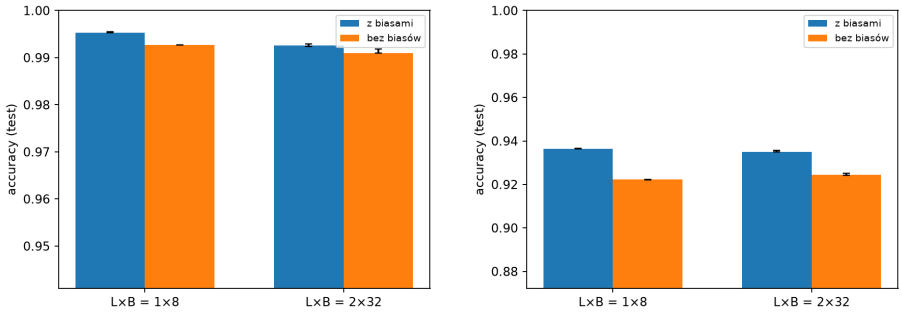
(a) *simple*, accuracy

(b) *three_ gauss*, accuracy

(c) *cube*, MAE w zakresie uczenia

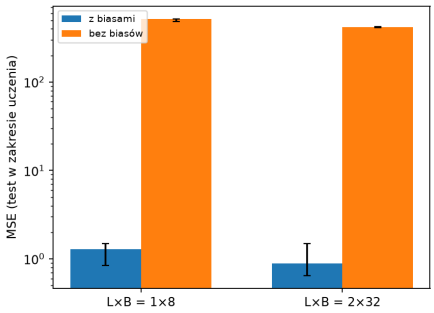
(d) *activation*, MAE w zakresie uczenia

Rysunek 11: Funkcja kosztu w zależności od współczynnika uczenia (sieć 2×32, tanh).

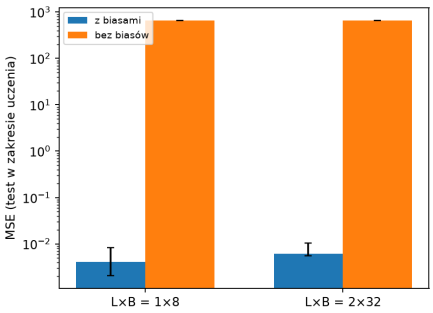


(a) *simple*

(b) *three_gauss*



(c) *cube*



(d) *activation*

Rysunek 12: Sieci 1×8 i 2×32 z biasami i bez biasów. W regresji MSE w zakresie uczenia (skala logarytmiczna).

6 Rozpoznawanie cyfr MNIST

6.1 Dane i procedura

Zbiór MNIST zawiera obrazy odręcznie pisanych cyfr 0–9 w skali szarości, o rozmiarze 28×28 pikseli: 60 000 uczących i 10 000 testowych. Każdy obraz traktujemy jako wektor 784 cech przeskalowanych do $[0, 1]$, a sieć ma 10 wyjść z funkcją softmax. Program i implementacja są te same co w części pierwszej; zmieniają się tylko parametry (tab. 3).

Sieć uczy się wyłącznie na zbiorze uczącym. Zbiór testowy oceniamy po każdej epoce tylko w celu zapisu wyników; nie wpływa on na uczenie ani na liczbę epok. Badania są analogiczne do części pierwszej: architektura ($L \in \{0, 1, 2, 3\}$, $B \in \{32, 128, 512\}$), funkcja aktywacji i funkcja kosztu dla współczynników uczenia $\{0{,}01;\ 0{,}03;\ 0{,}1;\ 0{,}3\}$ oraz biasy (sieci 1×128 i 2×128). Każdą konfigurację uczono z 5 ziarnami; wynik to średnia z ostatnich 5 epok, podawana jako mediana z ziaren.

warstwy ukryte	1×128
aktywacja warstw ukrytych	ReLU
wyjście i funkcja kosztu	softmax, entropia krzyżowa
biasy / inicjalizacja	tak / He, biasy zerowe
optymalizator	mini-batch SGD, batch 64
współczynnik uczenia	0,1 (stały)
liczba epok	20
normalizacja wejść	min-max do $[0, 1]$
precyzja obliczeń	32-bitowa

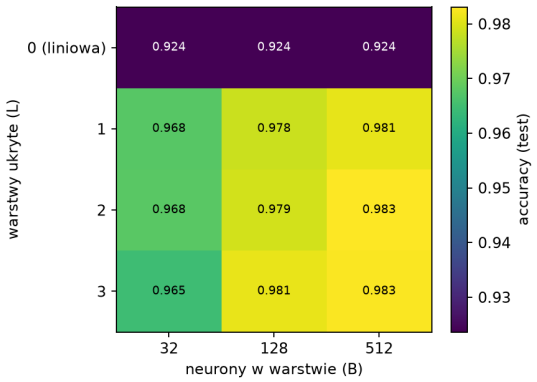
Tabela 3: Konfiguracja bazowa dla MNIST.

6.2 Architektura

Trafność rośnie przede wszystkim z szerokością warstw (rys. 13, tab. 4): od 92,4% dla sieci liniowej, przez 96,8% (1×32) i 97,8% (1×128), do 98,3% dla 2×512. Druga warstwa ukryta daje niewielki zysk, trzecia żaden. Sieci 2×512, 3×128 i 3×512 osiągają 100% trafności na zbiorze uczącym, a różnica między trafnością uczącą i testową rośnie z rozmiarem sieci (rys. 14b). Koszt na zbiorze uczącym maleje przez całe uczenie, natomiast koszt testowy po ok. 10 epokach prawie przestaje maleć (rys. 14a).

6.3 Funkcja aktywacji i funkcja kosztu

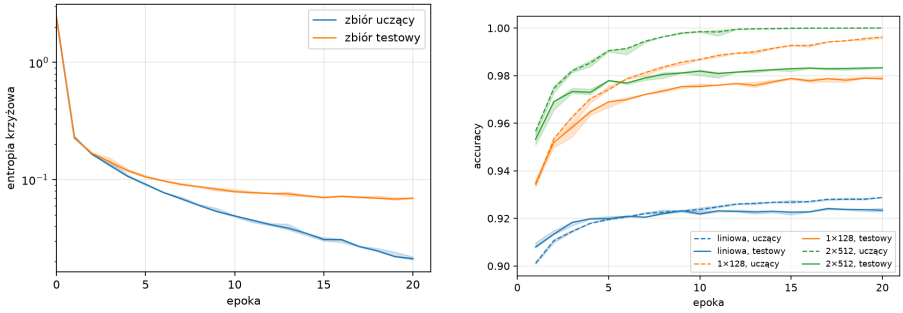
Dla wszystkich funkcji aktywacji trafność rośnie ze współczynnikiem uczenia w całym badanym zakresie (rys. 15a). Przy współczynniku 0,3 najlepsza jest ReLU (98,1%), przed tanh (97,9%) i sigmoidą (97,5%); przy współczynniku 0,01 przewaga ReLU nad sigmoidą wynosi ok. 4 p.p. Entropia krzyżowa jest lepsza od



Rysunek 13: MNIST: trafność na zbiorze testowym w zależności od liczby warstw ukrytych L i liczby neuronów w warstwie B.

architektura	liczba parametrów	accuracy (uczący)	accuracy (test)
liniowa	7 850	92,8%	92,4%
1×32	25 450	98,1%	96,8%
1×128	101 770	99,4%	97,8%
1×512	407 050	99,8%	98,1%
2×128	118 282	99,9%	97,9%
2×512	669 706	100%	98,3%
3×128	134 794	100%	98,1%
3×512	932 362	100%	98,3%

Tabela 4: MNIST: trafność wybranych architektur (mediana z 5 ziaren).

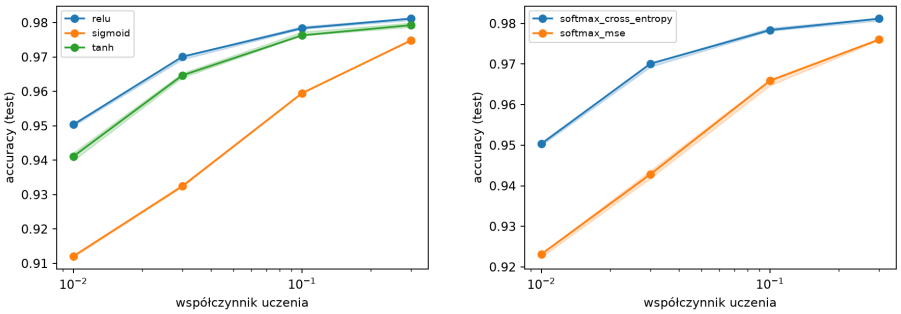


(a) koszt, sieć 1×128

(b) trafność, trzy architektury

Rysunek 14: MNIST: przebieg uczenia na zbiorze uczącym i testowym (mediana i IQR z 5 ziaren).

MSE przy każdym badanym kroku (rys. 15b), o 2,7 p.p. przy współczynnikach 0,01–0,03 i o 0,5 p.p. przy 0,3.

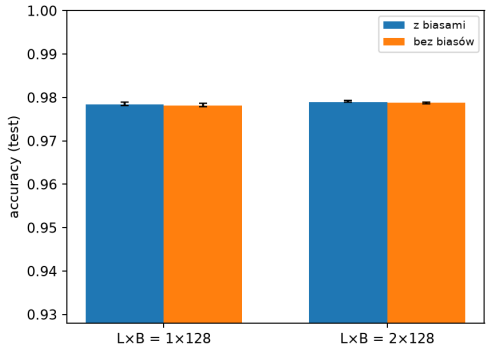


(a) funkcja aktywacji (b) funkcja kosztu

Rysunek 15: MNIST: trafność na zbiorze testowym w zależności od współczynnika uczenia (sieć 1×128).

6.4 Biasy

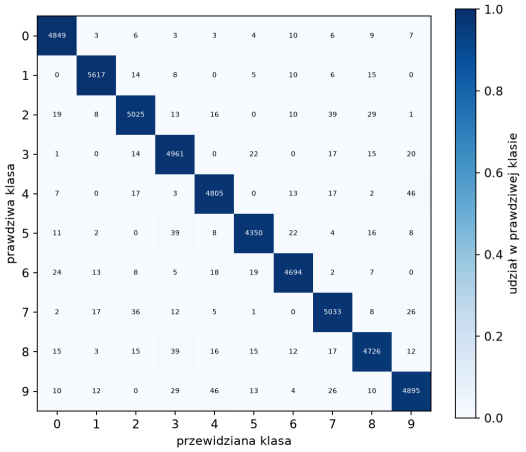
Na MNIST usunięcie biasów praktycznie nie zmienia wyniku (rys. 16): trafność sieci 1×128 wynosi 97,84% z biasami i 97,81% bez nich, a sieci 2×128 odpowiednio 97,90% i 97,88%. Różnice są mniejsze niż rozrzut między ziarnami.



Rysunek 16: MNIST: trafność na zbiorze testowym sieci z biasami i bez biasów.

6.5 Analiza błędów i wag

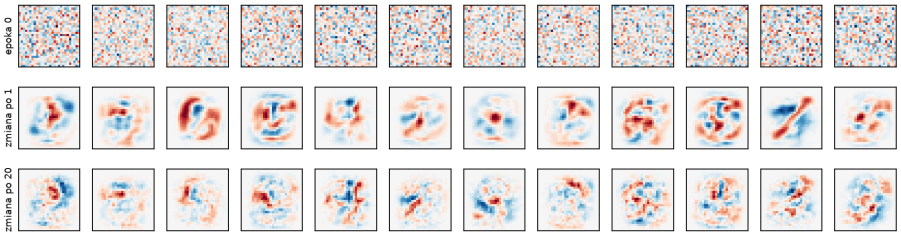
Macierz pomyłek sieci bazowej (rys. 17) pokazuje, że najczęściej mylone są pary 4 i 9 (po 46 przypadków w obie strony, sumarycznie dla 5 ziaren), 2 i 7 oraz



Rysunek 17: MNIST: macierz pomyłek sieci bazowej na zbiorze testowym po ostatniej epoce, zsumowana po 5 ziarnach.

cyfry 5 i 8 z cyfrą 3. Najlepiej rozpoznawane są cyfry 0 i 1 (ok. 99%), najsłabiej 5, 8 i 9 (ok. 97%).

Rysunek 18 przedstawia wagi pierwszej warstwy jako obrazy 28×28. Po pierwszej epoce zmiany wag tworzą gładkie wzory w środkowej części obrazu, w której leżą cyfry, a wagi pikseli brzegowych prawie się nie zmieniają. Niektóre wzory przypominają fragmenty cyfr, np. ukośną kreskę. Po 20 epokach wzory są drobniejsze i mniej regularne.



Rysunek 18: MNIST: wagi 12 pierwszych neuronów warstwy ukrytej sieci 1×128 (ziarno 0) przedstawione jako obrazy 28×28: wartości początkowe oraz ich zmiana po pierwszej i po ostatniej epoce (czerwony – dodatnie, niebieski – ujemne, każdy obraz we własnej skali).

7 Podsumowanie i wnioski

Zaimplementowana sieć MLP poprawnie rozwiązuje zadania klasyfikacji i regresji, a wyniki są powtarzalne: rozrzut między ziarnami jest mały, a uczenie rozbiegało się tylko w regresji przy największych współczynnikach uczenia. Z przeprowadzonych badań wynika, że:

- w regresji konieczna jest co najmniej jedna warstwa ukryta, natomiast w prostych zadaniach klasyfikacji w $\mathbb{R}^2$ warstwy ukryte dają niewielki zysk; dodatkowe warstwy nie poprawiają wyniku w zakresie danych uczących i pogarszają ekstrapolację,
- wpływ funkcji aktywacji zależy od współczynnika uczenia: sigmoida wymaga większego kroku, ale jest stabilna, a tanh i ReLU uczą się szybciej, lecz przy dużym kroku mogą się rozbiegać; poza zakresem danych uczących sieć przejmuje kształt funkcji aktywacji,
- w klasyfikacji entropia krzyżowa daje lepsze lub podobne wyniki niż MSE przy mniejszym kroku, a w regresji MSE jest lepsze od MAE, także w mierze MAE,
- biasy są niezbędne w regresji i poprawiają klasyfikację w $\mathbb{R}^2$, natomiast na MNIST nie mają zauważalnego wpływu,
- na MNIST najlepszą trafność (98,3%) osiąga sieć z dwiema warstwami po 512 neuronów; szerokość warstw ma większy wpływ niż ich liczba, a największe sieci dopasowują się w całości do zbioru uczącego.

Literatura

[1] X. Glorot, Y. Bengio, *Understanding the difficulty of training deep feedforward neural networks*, AISTATS, 2010.

[2] K. He, X. Zhang, S. Ren, J. Sun, *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*, ICCV, 2015.